Sprint -1 Foundation

SPRINT 1 — FOUNDATION

Build ALL database schemas + AI service + shared components

Estimated build time: 2-3 Replit sessions

PASTE THIS ENTIRE FILE INTO REPLIT AI AGENT

WHAT TO BUILD IN THIS SPRINT

1. Complete Prisma schema with ALL models for all 25 features
2. Multi-model AI service layer (Feature 12)
3. BullMQ job queue configuration
4. Shared AI UI components
5. Knowledge indexing pipeline backend (Feature 2 — indexing only, not search UI)

After this sprint, verify:

- `npx prisma migrate dev` runs without errors
- `npx prisma generate` creates client successfully
- AI service can call OpenAI and stream responses
- BullMQ worker starts and processes test jobs
- All shared components render in Storybook or test page

STEP 1: PRISMA SCHEMA

Add ALL of the following models to `prisma/schema.prisma`. These cover all 25 features.

```
// ============================================================
==
// prisma/schema.prisma — COMPLETE SCHEMA FOR ALL 25 AI FEATU
RES
// ============================================================
==

generator client {
  provider        = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider   = "postgresql"
  url        = env("DATABASE_URL")
  extensions = [pgvector(map: "vector", schema: "public"), uu
idOssp(map: "uuid-ossp")]
}

// ============================================================
==
// EXISTING MODELS (already built — do NOT recreate, just ref
erence)
// Add the new fields marked with "// NEW" to existing models
// ============================================================
==

// If these models already exist, just add the relation field
s below.
// If they don't exist exactly like this, adapt the new model
s to match your existing schema.
```

```
// model User { ... existing fields ... }
// model Workspace { ... existing fields ... }
// model Page { ... existing fields ... }
// model Task { ... existing fields ... }
// model Channel { ... existing fields ... }
// model Message { ... existing fields ... }
// model Meeting { ... existing fields ... }

// ============================================================
==
// FEATURE 1: AI WRITING ASSISTANT
// ============================================================
==

model AiWritingHistory {
  id             String    @id @default(uuid()) @db.Uuid
  userId         String    @db.Uuid
  workspaceId    String    @db.Uuid
  pageId         String?   @db.Uuid
  channelId      String?   @db.Uuid
  action         String    // 'write','improve','shorten','le
ngthen','tone','grammar','translate','summarize','brainstor
m','outline','continue','explain','action_items','custom'
  inputText      String?   @db.Text
  prompt         String    @db.Text
  outputText     String    @db.Text
  model          String    // 'gpt-4o','gpt-4o-mini','claude-
sonnet','gemini-pro'
  tone           String?   // 'professional','casual','friend
ly','academic','persuasive'
  language       String?   // target language for translation
  writerRole     String?   // 'marketer','engineer','executiv
e','designer','sales','support','hr','general'
  tokensUsed     Int       @default(0)
  accepted       Boolean   @default(false)
```

```
  feedbackRating Int?      // 1-5
  createdAt      DateTime  @default(now())

  @@index([userId, createdAt(sort: Desc)])
  @@index([workspaceId])
}

model BrandVoice {
  id              String   @id @default(uuid()) @db.Uuid
  workspaceId     String   @unique @db.Uuid
  defaultTone     String   @default("professional")
  styleGuideText  String?  @db.Text  // parsed content from u
ploaded style guide
  vocabularyRules Json?    // [{"use": "customers", "notUse":
"users"}, ...]
  formattingRules String?  @db.Text
  customRoles     Json?    // [{"name": "marketer", "systemPr
ompt": "..."}, ...]
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt
}

// ============================================================
==
// FEATURE 2: AI KNOWLEDGE MANAGER & SEARCH
// ============================================================
==

model KnowledgeIndex {
  id              String                    @id @default(uuid
()) @db.Uuid
  workspaceId     String                    @db.Uuid
  sourceType      String                    // 'page','tas
k','message','meeting_transcript','meeting_summary','commen
t','file'
  sourceId        String                    @unique @db.Uuid
```

```
  parentSourceId  String?                   @db.Uuid
  title           String
  content         String                   @db.Text
  contentChunks   Json?                    // [{chunkIndex:
number, text: string}]
  embedding       Unsupported("vector(1536)")?
  metadata        Json                     // {authorId,auth
orName,channelName,databaseName,pagePath,tags,createdDate,las
tModifiedDate}
  accessLevel     String                   @default("workspa
ce") // 'workspace','channel_members','page_viewers','privat
e'
  lastIndexedAt   DateTime                 @default(now())
  isStale         Boolean                  @default(false)
  createdAt       DateTime                 @default(now())
  updatedAt       DateTime                 @updatedAt

  @@index([workspaceId, sourceType])
  @@index([workspaceId, lastIndexedAt])
  @@index([workspaceId, accessLevel])
}

model KnowledgeQuery {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  query           String   @db.Text
  queryEmbedding  Unsupported("vector(1536)")?
  results         Json?    // [{sourceType,sourceId,title,sni
ppet,relevanceScore,url}]
  aiAnswer        String?  @db.Text
  citations       Json?    // [{sourceId,sourceType,title,sni
ppet}]
  responseTimeMs  Int?
  feedbackRating  Int?     // 1-5
  createdAt       DateTime @default(now())
```

```
  @@index([userId, createdAt(sort: Desc)])
  @@index([workspaceId])
}

model KnowledgeSuggestion {
  id               String   @id @default(uuid()) @db.Uuid
  workspaceId      String   @db.Uuid
  channelId        String?  @db.Uuid
  messageId        String?  @db.Uuid
  questionDetected String   @db.Text
  suggestedAnswer  String   @db.Text
  sources          Json     // [{sourceId,sourceType,title}]
  confidence       Float
  status           String   @default("suggested") // 'suggest
ed','accepted','dismissed','expired'
  shownToUserId    String?  @db.Uuid
  createdAt        DateTime @default(now())

  @@index([workspaceId, status])
  @@index([channelId])
}

model KnowledgeItem {
  id            String   @id @default(uuid()) @db.Uuid
  workspaceId   String   @db.Uuid
  category      String   // 'how_to','decision','policy','fa
q','reference'
  title         String
  content       String   @db.Text
  sourceType    String   // where this was extracted from
  sourceId      String   @db.Uuid
  confidence    Float    @default(0)
  status        String   @default("draft") // 'draft','publis
hed','archived'
  createdAt     DateTime @default(now())
```

```
  updatedAt     DateTime @updatedAt

  @@index([workspaceId, category])
  @@index([workspaceId, status])
}

// ============================================================
==
// FEATURE 3: AI AUTO-SCHEDULER & DAILY PLANNER
// ============================================================
==

model DailyPlan {
  id                 String   @id @default(uuid()) @db.Uuid
  userId             String   @db.Uuid
  workspaceId        String   @db.Uuid
  date               DateTime @db.Date
  status             String   @default("generated") // 'gener
ated','active','completed'
  timeBlocks         Json     // Array — see TimeBlock type b
elow
  aiSummary          Json?    // {totalPlannedMinutes,focusMi
nutes,meetingMinutes,mustDoCount,shouldDoCount,niceToDoCount,
topPriority,aiReasoning,completionLikelihood}
  completionRate     Float?
  userModifications  Int      @default(0)
  reviewNote         String?  @db.Text
  createdAt          DateTime @default(now())
  updatedAt          DateTime @updatedAt

  @@unique([userId, date])
  @@index([userId, date(sort: Desc)])
}

// TimeBlock JSON structure (stored in DailyPlan.timeBlocks a
rray):
```

```
// {
//   id: string (uuid),
//   type: 'task' | 'meeting' | 'focus' | 'break' | 'habit' |
'communication' | 'planning' | 'buffer' | 'custom',
//   title: string,
//   description?: string,
//   startTime: string ("09:00" — 24hr format),
//   endTime: string ("10:30"),
//   durationMinutes: number,
//   taskId?: string (uuid — linked task),
//   meetingId?: string (uuid — linked calendar event),
//   habitId?: string (uuid — linked habit),
//   priority: 'must_do' | 'should_do' | 'nice_to_do',
//   energyLevel: 'high' | 'medium' | 'low',
//   isFixed: boolean (meetings = true, tasks = false),
//   isCompleted: boolean,
//   completedAt?: string (ISO datetime),
//   isSkipped: boolean,
//   actualDurationMinutes?: number,
//   scheduledByAI: boolean,
//   aiReason?: string ("Scheduled in your peak focus window
10-12 AM")
// }

model SchedulingPreference {
  id                        String   @id @default(uuid()) @db.
Uuid
  userId                    String   @unique @db.Uuid
  workspaceId               String   @db.Uuid
  workingHours              Json     @default("{\"start\":\"0
9:00\",\"end\":\"18:00\"}")
  timezone                  String   @default("UTC")
  workDays                  Int[]    @default([1, 2, 3, 4, 5])
// 0=Sun, 1=Mon...6=Sat
  lunchTime                 Json     @default("{\"start\":\"1
2:30\",\"end\":\"13:30\"}")
```

```
  peakFocusWindow          Json?    // {"start":"10:00","en
d":"12:00"} — auto-learned
  preferMeetingsIn         String   @default("any") // 'morni
ng','afternoon','any'
  minFocusBlockMinutes     Int      @default(60)
  bufferBetweenMeetings    Int      @default(15) // minutes
  autoScheduleEnabled      Boolean  @default(true)
  morningPlanningEnabled   Boolean  @default(true)
  morningPlanningTime      String   @default("08:30")
  eveningReviewEnabled     Boolean  @default(true)
  eveningReviewTime        String   @default("17:30")
  planMode                 String   @default("suggest") // 'a
utomatic','suggest','manual'
  createdAt                DateTime @default(now())
  updatedAt                DateTime @updatedAt
}

// ============================================================
==
// FEATURE 4: AI AUTONOMOUS AGENTS
// ============================================================
==

model AiAgent {
  id                String    @id @default(uuid()) @db.Uuid
  workspaceId       String    @db.Uuid
  name              String
  description       String?   @db.Text
  icon              String?
  type              String    @default("custom") // 'built_i
n','custom','template'
  createdBy         String    @db.Uuid
  isShared          Boolean   @default(false)
  visibility        String    @default("workspace") // 'priva
te','workspace','specific_members'
  visibleToUserIds  String[]  @db.Uuid
```

```
  systemPrompt      String    @db.Text
  model             String    @default("gpt-4o")
  temperature       Float     @default(0.7)
  capabilities      String[]  // ['read_pages','write_page
s','read_tasks','write_tasks','read_messages','send_message
s','search_workspace','create_database_rows','update_database
_rows','send_notifications','read_calendar','web_search','gen
erate_content']
  dataAccess        Json      // {allPages:bool,specificPageI
ds:string[],allDatabases:bool,specificDatabaseIds:string[],al
lChannels:bool,specificChannelIds:string[],allMeetings:bool}
  triggerType       String    @default("manual") // 'manua
l','scheduled','event','message_in_channel','form_submissio
n','webhook'
  triggerConfig     Json?     // trigger-specific config
  isActive          Boolean   @default(true)
  maxRunsPerDay     Int       @default(100)
  maxActionsPerRun  Int       @default(25)
  timeoutMinutes    Int       @default(5)
  totalRuns         Int       @default(0)
  totalTokensUsed   Int       @default(0)
  lastRunAt         DateTime?
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
  deletedAt         DateTime?

  runs              AgentRun[]
  conversations     AgentConversation[]

  @@index([workspaceId, isActive])
  @@index([workspaceId, type])
  @@index([createdBy])
}

model AgentRun {
  id               String    @id @default(uuid()) @db.Uuid
```

```
  agentId          String    @db.Uuid
  workspaceId      String    @db.Uuid
  triggeredBy      String    // 'manual','schedule','even
t','message','form','webhook'
  triggerData      Json?
  invokedByUserId  String?   @db.Uuid
  status           String    @default("queued") // 'queue
d','running','completed','failed','cancelled','timeout'
  steps            Json[]    // [{stepNumber,action,input,out
put,durationMs,status:'success'|'failed'|'skipped',error?}]
  finalOutput      String?   @db.Text
  tokensUsed       Int       @default(0)
  durationMs       Int?
  error            String?   @db.Text
  startedAt        DateTime?
  completedAt      DateTime?
  createdAt        DateTime  @default(now())

  agent            AiAgent   @relation(fields: [agentId], ref
erences: [id])

  @@index([agentId, createdAt(sort: Desc)])
  @@index([workspaceId])
  @@index([status])
}

model AgentConversation {
  id        String   @id @default(uuid()) @db.Uuid
  agentId   String   @db.Uuid
  userId    String   @db.Uuid
  messages  Json[]   // [{role:'user'|'agent',content:string,
timestamp:string,runId?:string}]
  status    String   @default("active") // 'active','closed'
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
```

```
  agent     AiAgent  @relation(fields: [agentId], references:
[id])

  @@index([agentId, userId])
  @@index([userId, updatedAt(sort: Desc)])
}

// ============================================================
==
// FEATURE 5: AI PROJECT MANAGER
// ============================================================
==

model ProjectStatusReport {
  id                String   @id @default(uuid()) @db.Uuid
  workspaceId       String   @db.Uuid
  databaseId        String   @db.Uuid // which task database/
project
  generatedBy       String   // 'ai_auto','ai_on_demand','man
ual'
  reportType        String   // 'daily_standup','weekly_statu
s','sprint_review','risk_report'
  periodStart       DateTime @db.Date
  periodEnd         DateTime @db.Date
  content           Json     // {summary,completedTasks[],inP
rogressTasks[],blockedTasks[],overdueTasks[],upcomingDeadline
s[],risks[],velocity,prediction,highlights[],concerns[]}
  postedToChannelId String?  @db.Uuid
  postedMessageId   String?  @db.Uuid
  createdAt         DateTime @default(now())

  @@index([workspaceId, databaseId])
  @@index([workspaceId, reportType])
}

model AiRiskAlert {
```

```
  id                String    @id @default(uuid()) @db.Uuid
  workspaceId       String    @db.Uuid
  databaseId        String    @db.Uuid
  riskType          String    // 'deadline_at_risk','team_memb
er_overloaded','blocked_task_chain','scope_creep','velocity_d
eclining','unassigned_urgent','dependency_conflict'
  severity          String    // 'low','medium','high','critic
al'
  title             String
  description       String    @db.Text
  affectedTaskIds   String[]  @db.Uuid
  affectedUserIds   String[]  @db.Uuid
  suggestedAction   String    @db.Text
  status            String    @default("active") // 'activ
e','acknowledged','resolved','dismissed'
  acknowledgedBy    String?   @db.Uuid
  resolvedAt        DateTime?
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt

  @@index([workspaceId, status])
  @@index([workspaceId, severity])
}

model AiWorkflowAutomation {
  id                String    @id @default(uuid()) @db.Uuid
  workspaceId       String    @db.Uuid
  name              String
  description       String?   @db.Text
  createdFromPrompt String?  @db.Text // natural language inp
ut
  trigger           Json      // {type:'task_created'|'task_st
atus_changed'|'task_overdue'|'scheduled'|...,config:{}}
  conditions        Json[]    // [{field,operator,value}]
  actions           Json[]    // [{type:'update_task'|'send_me
ssage'|'create_task'|'assign'|'notify',config:{}}]
```

```
  isActive         Boolean   @default(true)
  runCount         Int       @default(0)
  lastRunAt        DateTime?
  createdBy        String    @db.Uuid
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt
  deletedAt        DateTime?

  @@index([workspaceId, isActive])
}

// ============================================================
==
// FEATURE 7: AI MEETING INTELLIGENCE
// ============================================================
==

model MeetingIntelligence {
  id                String   @id @default(uuid()) @db.Uuid
  meetingId         String   @unique @db.Uuid // FK to existi
ng Meeting model
  workspaceId       String   @db.Uuid
  audioUrl          String?  // S3/R2 URL of recording
  audioDurationSecs Int?
  transcriptRaw     String?  @db.Text // full transcription
  transcriptSegments Json?   // [{speakerLabel,startTime,endT
ime,text}]
  speakerLabels     Json?    // [{label:"Speaker 1",userId?:s
tring,name?:string}]
  aiSummary         String?  @db.Text
  keyPoints         Json?    // string[]
  decisions         Json?    // [{decision:string,context:str
ing,decidedBy?:string}]
  actionItems       Json?    // [{title,description,suggested
Owner?,suggestedDueDate?,taskCreated:bool,taskId?}]
  openQuestions     Json?    // string[]
```

```
  prepBrief         String?  @db.Text
  processingStatus  String   @default("pending") // 'pendin
g','transcribing','summarizing','completed','failed'
  processingError   String?
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  @@index([workspaceId])
  @@index([processingStatus])
}

// ============================================================
==
// FEATURE 8: AI DATABASE INTELLIGENCE
// ============================================================
==

model AiAutoFillColumn {
  id           String   @id @default(uuid()) @db.Uuid
  workspaceId  String   @db.Uuid
  databaseId   String   @db.Uuid // which task database this
column belongs to
  columnName   String
  aiPrompt     String   @db.Text // "Based on {Title} and {De
scription}, assign category: Bug, Feature, Enhancement"
  inputFields  String[] // ['title', 'description'] — which f
ields the AI reads
  outputType   String   @default("text") // 'text','selec
t','number'
  selectOptions String[]? // if outputType='select', the allo
wed values
  isActive     Boolean  @default(true)
  createdBy    String   @db.Uuid
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
```

```
  @@index([workspaceId, databaseId])
}

// ============================================================
==
// FEATURE 9: AI FOCUS & PRODUCTIVITY COACH
// ============================================================
==

model FocusSession {
  id                String    @id @default(uuid()) @db.Uuid
  userId            String    @db.Uuid
  workspaceId       String    @db.Uuid
  taskId            String?   @db.Uuid // linked task
  title             String    // what the user is focusing on
  plannedMinutes    Int
  actualMinutes     Int?
  startedAt         DateTime
  endedAt           DateTime?
  pausedMinutes     Int       @default(0) // total time paused
  status            String    @default("active") // 'activ
e','paused','completed','abandoned'
  notificationsPaused Boolean @default(true)
  queuedNotificationCount Int @default(0) // notifications he
ld during session
  createdAt         DateTime  @default(now())

  @@index([userId, createdAt(sort: Desc)])
  @@index([userId, status])
}

model FocusTimeBlock {
  id                 String   @id @default(uuid()) @db.Uuid
  userId             String   @db.Uuid
  workspaceId        String   @db.Uuid
  date               DateTime @db.Date
```

```
  startTime         String   // "10:00"
  endTime           String   // "12:00"
  durationMinutes   Int
  calendarEventId   String?  // Google Calendar event ID (syn
ced)
  isProtected       Boolean  @default(true) // shows as "bus
y" on calendar
  source            String   @default("ai") // 'ai','manual'
  createdAt         DateTime @default(now())

  @@index([userId, date])
}

model ProductivityScore {
  id                  String   @id @default(uuid()) @db.Uuid
  userId              String   @db.Uuid
  workspaceId         String   @db.Uuid
  date                DateTime @db.Date
  focusMinutes        Int      @default(0)
  meetingMinutes      Int      @default(0)
  tasksCompleted      Int      @default(0)
  tasksPlanned        Int      @default(0)
  habitsCompleted     Int      @default(0)
  habitsPlanned       Int      @default(0)
  interruptionCount   Int      @default(0)
  productivityScore   Float?   // 0-100 AI-calculated
  aiInsight           String?  @db.Text // "You focused 2h mo
re than last week. Peak: 10-11 AM."
  createdAt           DateTime @default(now())

  @@unique([userId, date])
  @@index([userId, date(sort: Desc)])
}

// ============================================================
==
```

```
// FEATURE 13: AI EMAIL ASSISTANT
// ============================================================
==

model EmailAccount {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  provider        String   // 'gmail','outlook'
  email           String
  accessToken     String   @db.Text // encrypted
  refreshToken    String?  @db.Text // encrypted
  syncEnabled     Boolean  @default(true)
  lastSyncAt      DateTime?
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@unique([userId, email])
}

model EmailThread {
  id              String   @id @default(uuid()) @db.Uuid
  emailAccountId  String   @db.Uuid
  externalId      String   // provider thread ID
  subject         String
  participants    Json     // [{name,email,role:'from'|'t
o'|'cc'}]
  latestSnippet   String?  @db.Text
  messageCount    Int      @default(1)
  isRead          Boolean  @default(false)
  aiPriority      String?  // 'urgent','important','fyi','lo
w'
  aiLabels        String[] // ['action_required','waiting','i
nformational','scheduled']
  aiSummary       String?  @db.Text
  lastMessageAt   DateTime
```

```
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@index([emailAccountId, lastMessageAt(sort: Desc)])
  @@index([emailAccountId, aiPriority])
}

// ============================================================
==
// FEATURE 14: AI HABIT TRACKER & STREAK ENGINE
// ============================================================
==

model Habit {
  id                  String    @id @default(uuid()) @db.Uuid
  userId              String    @db.Uuid
  workspaceId         String    @db.Uuid
  name                String
  icon                String?   // emoji
  color               String?   // hex
  durationMinutes     Int
  frequency           String    @default("daily") // 'dail
y','weekdays','custom'
  customDays          Int[]     // [1,2,3,4,5] for weekdays
  preferredTimeRange  Json?     // {"earliest":"09:00","lates
t":"17:00"}
  priority            String    @default("medium") // 'hig
h','medium','low'
  isFlexible          Boolean   @default(true)
  isProtected         Boolean   @default(false) // block on s
hared calendar
  streakCurrent       Int       @default(0)
  streakLongest       Int       @default(0)
  isActive            Boolean   @default(true)
  goalId              String?   @db.Uuid // linked to Feature
15 goal
```

```
  createdAt           DateTime  @default(now())
  updatedAt           DateTime  @updatedAt
  deletedAt           DateTime?

  completions         HabitCompletion[]

  @@index([userId, isActive])
}

model HabitCompletion {
  id          String   @id @default(uuid()) @db.Uuid
  habitId     String   @db.Uuid
  userId      String   @db.Uuid
  date        DateTime @db.Date
  completedAt DateTime @default(now())

  habit       Habit    @relation(fields: [habitId], reference
s: [id])

  @@unique([habitId, date])
  @@index([userId, date(sort: Desc)])
}

// ============================================================
==
// FEATURE 15: AI GOAL & OKR SYSTEM
// ============================================================
==

model Goal {
  id              String    @id @default(uuid()) @db.Uuid
  workspaceId     String    @db.Uuid
  title           String
  description     String?   @db.Text
  ownerId         String    @db.Uuid // user who owns this go
al
```

```
  level           String    // 'company','team','individual'
  parentGoalId    String?   @db.Uuid // hierarchical: company
> team > individual
  status          String    @default("on_track") // 'on_trac
k','at_risk','behind','completed','cancelled'
  progressPercent Float     @default(0) // 0-100, auto-calcul
ated from key results
  startDate       DateTime  @db.Date
  targetDate      DateTime  @db.Date
  linkedDatabaseIds String[] @db.Uuid // task databases that
contribute to this goal
  aiRiskAssessment String?  @db.Text
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime?

  keyResults      KeyResult[]

  @@index([workspaceId, level])
  @@index([workspaceId, status])
  @@index([ownerId])
  @@index([parentGoalId])
}

model KeyResult {
  id              String   @id @default(uuid()) @db.Uuid
  goalId          String   @db.Uuid
  title           String
  metricType      String   // 'number','percentage','currenc
y','boolean'
  currentValue    Float    @default(0)
  targetValue     Float
  startValue      Float    @default(0)
  unit            String?  // '%', '$', 'users', 'tickets', e
tc.
  progressPercent Float    @default(0) // auto-calculated: (c
```

```
urrent-start)/(target-start)*100
  autoTrackSource String?  // 'task_count','task_completio
n','habit_streak','manual'
  autoTrackConfig Json?    // {databaseId?,statusFilter?,habi
tId?}
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  goal            Goal     @relation(fields: [goalId], refere
nces: [id], onDelete: Cascade)

  @@index([goalId])
}

// ============================================================
==
// FEATURE 16: AI CALENDAR OPTIMIZER
// ============================================================
==

model CalendarOptimizationRule {
  id                    String   @id @default(uuid()) @db.Uui
d
  workspaceId           String   @db.Uuid
  ruleType              String   // 'no_meeting_window','buff
er_time','meeting_defrag','focus_protection'
  config                Json     // depends on ruleType
  // no_meeting_window: {days:[2,4],startTime:"10:00",endTim
e:"12:00"}
  // buffer_time: {minutes:15,applyBetween:'all'|'external_on
ly'}
  // meeting_defrag: {enabled:true,preferMorning:true}
  // focus_protection: {minBlockMinutes:60,dailyTargetMinute
s:120}
  scope                 String   @default("workspace") // 'wo
rkspace','team','individual'
```

```
  scopeId               String?  @db.Uuid // teamId or userId
  isActive              Boolean  @default(true)
  createdBy             String   @db.Uuid
  createdAt             DateTime @default(now())
  updatedAt             DateTime @updatedAt

  @@index([workspaceId, ruleType])
}

// ============================================================
==
// FEATURE 17: AI TEMPLATE ENGINE
// ============================================================
==

model Template {
  id            String   @id @default(uuid()) @db.Uuid
  workspaceId   String?  @db.Uuid // null = global/public tem
plate
  name          String
  description   String?  @db.Text
  category      String   // 'project_management','persona
l','marketing','engineering','hr','sales','design','custom'
  type          String   // 'page','database','workspace','wo
rkflow','agent'
  content       Json     // full template content (page block
s, database schema, etc.)
  thumbnail     String?  // URL to preview image
  isPublic      Boolean  @default(false) // visible in global
gallery
  usageCount    Int      @default(0)
  rating        Float    @default(0)
  tags          String[]
  createdBy     String   @db.Uuid
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
```

```
  deletedAt     DateTime?

  @@index([workspaceId])
  @@index([category, isPublic])
  @@index([usageCount(sort: Desc)])
}

// ============================================================
==
// FEATURE 18: AI VOICE COMMANDS & NOTES
// ============================================================
==

model VoiceNote {
  id             String    @id @default(uuid()) @db.Uuid
  userId         String    @db.Uuid
  workspaceId    String    @db.Uuid
  audioUrl       String    // S3/R2 URL
  durationSecs   Int
  transcript     String?   @db.Text
  aiSummary      String?   @db.Text
  convertedTo    String?   // 'task','page','meeting_note',nu
ll
  convertedId    String?   @db.Uuid // ID of created task/pag
e
  status         String    @default("recording") // 'recordin
g','transcribing','completed','failed'
  createdAt      DateTime  @default(now())

  @@index([userId, createdAt(sort: Desc)])
}

// ============================================================
==
// FEATURE 19: AI WORKFLOW RECORDER & PLAYBACK
// ============================================================
```

```
==

model Workflow {
  id              String    @id @default(uuid()) @db.Uuid
  workspaceId     String    @db.Uuid
  name            String
  description     String?   @db.Text
  recordedSteps   Json[]    // [{stepNumber,actionType:'creat
e_task'|'send_message'|'create_page'|...,rawData:{},cleanedDa
ta:{},variables:[]}]
  variables       Json[]    // [{name:'project_name',type:'te
xt'|'user'|'date'|'database',defaultValue?}]
  isPublic        Boolean   @default(false) // shared with wo
rkspace
  runCount        Int       @default(0)
  triggerType     String    @default("manual") // 'manual','s
cheduled','event'
  triggerConfig   Json?
  createdBy       String    @db.Uuid
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime?

  @@index([workspaceId])
  @@index([createdBy])
}

model WorkflowRun {
  id           String    @id @default(uuid()) @db.Uuid
  workflowId   String    @db.Uuid
  userId       String    @db.Uuid
  variableValues Json    // filled-in variable values
  steps        Json[]    // [{stepNumber,status:'success'|'fa
iled'|'skipped',output?,error?}]
  status       String    @default("running") // 'running','co
mpleted','failed'
```

```
  startedAt    DateTime  @default(now())
  completedAt  DateTime?

  @@index([workflowId, startedAt(sort: Desc)])
}

// ============================================================
==
// FEATURE 20: AI SMART NOTIFICATIONS
// ============================================================
==

model NotificationPreference {
  id                   String   @id @default(uuid()) @db.Uuid
  userId               String   @unique @db.Uuid
  workspaceId          String   @db.Uuid
  alwaysNotify         String[] // ['direct_mention','direct_
message','task_assigned','deadline_today']
  batchNotify          String[] // ['channel_message','task_u
pdated','comment_added']
  muteNotify           String[] // ['channel_joined','reactio
n_added']
  batchFrequency       String   @default("3_per_day") // '2_p
er_day','3_per_day','hourly'
  batchTimes           String[] @default(["09:00", "13:00",
"17:00"])
  focusModeAction      String   @default("queue") // 'queu
e','urgent_only','all'
  aiTriageEnabled      Boolean  @default(true)
  createdAt            DateTime @default(now())
  updatedAt            DateTime @updatedAt
}

model NotificationItem {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
```

```
  workspaceId     String   @db.Uuid
  type            String   // 'mention','dm','task_assigne
d','task_due','comment','reaction','channel_activity','agent_
result','risk_alert','goal_update','system'
  title           String
  body            String   @db.Text
  sourceType      String?  // 'message','task','meeting','age
nt','goal','system'
  sourceId        String?  @db.Uuid
  aiPriority      String   @default("normal") // 'urgent','im
portant','normal','low'
  isRead          Boolean  @default(false)
  isBatched       Boolean  @default(false) // part of a batch
digest
  batchId         String?  @db.Uuid
  deliveredAt     DateTime?
  readAt          DateTime?
  createdAt       DateTime @default(now())

  @@index([userId, isRead, createdAt(sort: Desc)])
  @@index([userId, aiPriority])
}

// ==========================================================
==
// FEATURE 21: AI TIME TRACKING
// ==========================================================
==

model TimeEntry {
  id              String    @id @default(uuid()) @db.Uuid
  userId          String    @db.Uuid
  workspaceId     String    @db.Uuid
  taskId          String?   @db.Uuid
  focusSessionId  String?   @db.Uuid // linked to FocusSessio
n
```

```
  description     String?
  durationMinutes Int
  startedAt       DateTime
  endedAt         DateTime?
  source          String    @default("manual") // 'manual','f
ocus_session','auto_detected'
  createdAt       DateTime  @default(now())

  @@index([userId, startedAt(sort: Desc)])
  @@index([taskId])
}

model TaskDurationEstimate {
  id                String   @id @default(uuid()) @db.Uuid
  taskId            String   @db.Uuid
  workspaceId       String   @db.Uuid
  estimatedMinutes  Int
  actualMinutes     Int?
  confidence        Float    @default(0.5) // 0-1
  estimatedByAi     Boolean  @default(true)
  adjustedByUser    Boolean  @default(false)
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  @@unique([taskId])
}

// ============================================================
==
// FEATURE 22: AI TEAM COLLABORATION INSIGHTS
// ============================================================
==

model TeamInsightSnapshot {
  id                  String   @id @default(uuid()) @db.Uuid
  workspaceId         String   @db.Uuid
```

```
  date                DateTime @db.Date
  memberStats         Json     // [{userId,focusMinutes,meeti
ngMinutes,tasksCompleted,activeTaskCount,overdueTasks,workloa
dScore:'green'|'yellow'|'red'}]
  teamMetrics         Json     // {avgFocusMinutes,avgMeeting
Minutes,avgTasksCompleted,meetingFreeCompliance,collaboration
Patterns}
  aiInsights          Json     // [{type:'burnout_risk'|'over
load'|'recommendation',userId?,message}]
  createdAt           DateTime @default(now())

  @@unique([workspaceId, date])
  @@index([workspaceId, date(sort: Desc)])
}

// ============================================================
==
// FEATURE 24: AI INTEGRATION HUB
// ============================================================
==

model ExternalIntegration {
  id                  String    @id @default(uuid()) @db.Uuid
  workspaceId         String    @db.Uuid
  type                String    // 'google_drive','github','g
itlab','jira','linear','slack','figma','confluence'
  displayName         String    // "Acme Corp Google Drive"
  accessTokenEncrypted String   @db.Text
  refreshTokenEncrypted String? @db.Text
  scopes              String[]
  syncEnabled         Boolean   @default(true)
  syncStatus          String    @default("active") // 'activ
e','syncing','error','disconnected'
  lastSyncAt          DateTime?
  documentCount       Int       @default(0)
  errorMessage        String?
```

```
  createdBy           String    @db.Uuid
  createdAt           DateTime  @default(now())
  updatedAt           DateTime  @updatedAt

  @@index([workspaceId, type])
}

// ============================================================
==
// FEATURE 25: AI ONBOARDING ASSISTANT
// ============================================================
==

model OnboardingProgress {
  id                  String   @id @default(uuid()) @db.Uuid
  userId              String   @unique @db.Uuid
  workspaceId         String   @db.Uuid
  completedSteps      String[] // ['welcome','profile','first
_task','first_page','join_channel','first_habit','ai_searc
h','daily_plan']
  currentStep         String?
  isComplete          Boolean  @default(false)
  dismissedAt         DateTime?
  featureSuggestions  Json?    // [{feature,reason,suggested:
bool,dismissed:bool}] — AI suggests unused features
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt
}

// ============================================================
==
// AI USAGE TRACKING (shared across all features)
// ============================================================
==

model AiUsageLog {
```

```
  id            String   @id @default(uuid()) @db.Uuid
  userId        String   @db.Uuid
  workspaceId   String   @db.Uuid
  feature       String   // 'writing','search','planner','age
nt','project_manager','task','meeting','database','focus','do
c_gen','messaging','email','voice','template'
  action        String   // specific action within feature
  model         String   // which AI model was used
  inputTokens   Int      @default(0)
  outputTokens  Int      @default(0)
  durationMs    Int?
  success       Boolean  @default(true)
  error         String?
  createdAt     DateTime @default(now())

  @@index([workspaceId, feature, createdAt])
  @@index([userId, createdAt(sort: Desc)])
}
```

Run `npx prisma migrate dev --name add_all_ai_features` after adding all models.

STEP 2: MULTI-MODEL AI SERVICE (Feature 12)

Create the centralized AI service that ALL other features use. This is the most critical file.

Create file: `src/lib/ai/ai-service.ts`

```
// src/lib/ai/ai-service.ts
// CENTRAL AI SERVICE — Every AI feature calls this, never Op
enAI directly
// Provider-agnostic: swap providers by changing config only

import OpenAI from "openai";

// ============================================================
```

```
==
// TYPES
// ============================================================
==

export type AiModel = "gpt-4o" | "gpt-4o-mini" | "claude-sonn
et" | "gemini-pro";
export type AiProvider = "openai" | "anthropic" | "google";

export interface AiGenerateOptions {
  model?: AiModel;
  maxTokens?: number;
  temperature?: number;
  systemPrompt?: string;
  stream?: boolean;
}

export interface AiStreamChunk {
  text: string;
  done: boolean;
}

export interface AiEmbeddingResult {
  embedding: number[];
  tokensUsed: number;
}

interface ModelConfig {
  provider: AiProvider;
  modelId: string;       // actual model ID sent to provider
  maxTokens: number;
  costPer1kInput: number;
  costPer1kOutput: number;
}

// ============================================================
```

```
==
// MODEL REGISTRY
// ============================================================
==

const MODEL_REGISTRY: Record<AiModel, ModelConfig> = {
  "gpt-4o": {
    provider: "openai",
    modelId: "gpt-4o",
    maxTokens: 128000,
    costPer1kInput: 0.005,
    costPer1kOutput: 0.015,
  },
  "gpt-4o-mini": {
    provider: "openai",
    modelId: "gpt-4o-mini",
    maxTokens: 128000,
    costPer1kInput: 0.00015,
    costPer1kOutput: 0.0006,
  },
  "claude-sonnet": {
    provider: "anthropic",
    modelId: "claude-sonnet-4-20250514",
    maxTokens: 200000,
    costPer1kInput: 0.003,
    costPer1kOutput: 0.015,
  },
  "gemini-pro": {
    provider: "google",
    modelId: "gemini-1.5-pro",
    maxTokens: 1000000,
    costPer1kInput: 0.00125,
    costPer1kOutput: 0.005,
  },
};
```

```
// Auto-routing: best model per task type
const AUTO_MODEL_MAP: Record<string, AiModel> = {
  writing: "gpt-4o",        // Good all-round writing
  code: "gpt-4o",           // Strong at code
  analysis: "gpt-4o",       // Data analysis
  summary: "gpt-4o-mini",   // Summaries are simpler
  quick: "gpt-4o-mini",     // Fast, cheap tasks
  creative: "gpt-4o",       // Creative writing
  translation: "gpt-4o",    // Multi-language
};

// ============================================================
==
// AI SERVICE CLASS
// ============================================================
==

class AiService {
  private openai: OpenAI;

  constructor() {
    this.openai = new OpenAI({
      apiKey: process.env.OPENAI_API_KEY,
    });
  }

  /**
   * Select the best model for a task type, or use the specif
ied model
   */
  resolveModel(taskType?: string, requestedModel?: AiModel):
AiModel {
    if (requestedModel) return requestedModel;
    if (taskType && AUTO_MODEL_MAP[taskType]) return AUTO_MOD
EL_MAP[taskType];
    return "gpt-4o"; // default
```

```
  }

  /**
   * Generate text (non-streaming). Use for short responses.
   */
  async generateText(
    userPrompt: string,
    options: AiGenerateOptions = {}
  ): Promise<{ text: string; tokensUsed: { input: number; out
put: number } }> {
    const model = options.model || "gpt-4o";
    const config = MODEL_REGISTRY[model];

    // Currently only OpenAI implemented. Add Anthropic/Googl
e providers here later.
    const messages: OpenAI.Chat.ChatCompletionMessageParam[]
= [];
    if (options.systemPrompt) {
      messages.push({ role: "system", content: options.system
Prompt });
    }
    messages.push({ role: "user", content: userPrompt });

    const response = await this.openai.chat.completions.creat
e({
      model: config.modelId,
      messages,
      max_tokens: options.maxTokens || 2000,
      temperature: options.temperature ?? 0.7,
    });

    const text = response.choices[0]?.message?.content || "";
    const usage = response.usage;

    return {
      text,
```

```
      tokensUsed: {
        input: usage?.prompt_tokens || 0,
        output: usage?.completion_tokens || 0,
      },
    };
  }

  /**
   * Generate text with streaming. Use for all user-facing AI
responses.
   * Returns an async generator that yields text chunks.
   */
  async *generateStream(
    userPrompt: string,
    options: AiGenerateOptions = {}
  ): AsyncGenerator<AiStreamChunk> {
    const model = options.model || "gpt-4o";
    const config = MODEL_REGISTRY[model];

    const messages: OpenAI.Chat.ChatCompletionMessageParam[]
= [];
    if (options.systemPrompt) {
      messages.push({ role: "system", content: options.system
Prompt });
    }
    messages.push({ role: "user", content: userPrompt });

    const stream = await this.openai.chat.completions.create
({
      model: config.modelId,
      messages,
      max_tokens: options.maxTokens || 2000,
      temperature: options.temperature ?? 0.7,
      stream: true,
    });
```

```
    for await (const chunk of stream) {
      const text = chunk.choices[0]?.delta?.content || "";
      const done = chunk.choices[0]?.finish_reason === "sto
p";
      if (text || done) {
        yield { text, done };
      }
    }
  }

  /**
   * Generate text with function/tool calling. Used by AI Age
nts (Feature 4).
   */
  async generateWithTools(
    messages: OpenAI.Chat.ChatCompletionMessageParam[],
    tools: OpenAI.Chat.ChatCompletionTool[],
    options: AiGenerateOptions = {}
  ): Promise<OpenAI.Chat.ChatCompletionMessage> {
    const model = options.model || "gpt-4o";
    const config = MODEL_REGISTRY[model];

    const response = await this.openai.chat.completions.creat
e({
      model: config.modelId,
      messages,
      tools,
      max_tokens: options.maxTokens || 4000,
      temperature: options.temperature ?? 0.5,
    });

    return response.choices[0].message;
  }

  /**
   * Generate embedding vector for semantic search. Used by F
```

```
eature 2 (Knowledge).
   */
  async generateEmbedding(text: string): Promise<AiEmbeddingR
esult> {
    const response = await this.openai.embeddings.create({
      model: "text-embedding-3-small",
      input: text,
      dimensions: 1536,
    });

    return {
      embedding: response.data[0].embedding,
      tokensUsed: response.usage.total_tokens,
    };
  }

  /**
   * Generate embeddings in batch (up to 20 texts). Used by K
nowledge Indexer.
   */
  async generateEmbeddingBatch(texts: string[]): Promise<AiEm
beddingResult[]> {
    const response = await this.openai.embeddings.create({
      model: "text-embedding-3-small",
      input: texts,
      dimensions: 1536,
    });

    return response.data.map((item, i) => ({
      embedding: item.embedding,
      tokensUsed: Math.ceil(response.usage.total_tokens / tex
ts.length),
    }));
  }

  /**
```

```
   * Generate structured JSON output. Used by Task Breakdown,
Database AI, etc.
   */
  async generateJSON<T>(
    userPrompt: string,
    options: AiGenerateOptions & { jsonSchema?: string } = {}
  ): Promise<T> {
    const systemPrompt = `${options.systemPrompt || ""}\n\nIM
PORTANT: Respond with valid JSON only. No markdown, no backti
cks, no explanation. Just the JSON object.`;

    const { text } = await this.generateText(userPrompt, {
      ...options,
      systemPrompt,
      temperature: options.temperature ?? 0.3, // lower temp
for structured output
    });

    // Clean potential markdown backticks
    const cleaned = text.replace(/```json\n?/g, "").replace(/
```\n?/g, "").trim();
    return JSON.parse(cleaned) as T;
  }
}

// Singleton export
export const aiService = new AiService();
```

**Create file:** `src/lib/ai/embeddings.ts`

```
// src/lib/ai/embeddings.ts
// Embedding generation with Redis caching

import { aiService } from "./ai-service";
import { redis } from "../redis";
import crypto from "crypto";
```
```

```
const CACHE_TTL = 86400; // 24 hours
const CACHE_PREFIX = "emb:";

function hashText(text: string): string {
  return crypto.createHash("sha256").update(text).digest("he
x").slice(0, 16);
}

/**
 * Get embedding for text, with Redis cache
 */
export async function getEmbedding(text: string): Promise<num
ber[]> {
  const cacheKey = `${CACHE_PREFIX}${hashText(text)}`;

  // Check cache
  const cached = await redis.get(cacheKey);
  if (cached) {
    return JSON.parse(cached as string);
  }

  // Generate new embedding
  const result = await aiService.generateEmbedding(text);

  // Cache it
  await redis.setex(cacheKey, CACHE_TTL, JSON.stringify(resul
t.embedding));

  return result.embedding;
}

/**
 * Get embeddings for multiple texts in batch, with caching
 */
export async function getEmbeddingBatch(texts: string[]): Pro
```

```
mise<number[][]> {
  const results: (number[] | null)[] = new Array(texts.lengt
h).fill(null);
  const uncachedIndices: number[] = [];
  const uncachedTexts: string[] = [];

  // Check cache for each text
  for (let i = 0; i < texts.length; i++) {
    const cacheKey = `${CACHE_PREFIX}${hashText(texts[i])}`;
    const cached = await redis.get(cacheKey);
    if (cached) {
      results[i] = JSON.parse(cached as string);
    } else {
      uncachedIndices.push(i);
      uncachedTexts.push(texts[i]);
    }
  }

  // Generate embeddings for uncached texts (batch of 20 max)
  if (uncachedTexts.length > 0) {
    for (let batch = 0; batch < uncachedTexts.length; batch +
= 20) {
      const batchTexts = uncachedTexts.slice(batch, batch + 2
0);
      const batchResults = await aiService.generateEmbeddingB
atch(batchTexts);

      for (let j = 0; j < batchResults.length; j++) {
        const originalIndex = uncachedIndices[batch + j];
        results[originalIndex] = batchResults[j].embedding;

        // Cache
        const cacheKey = `${CACHE_PREFIX}${hashText(batchText
s[j])}`;
        await redis.setex(cacheKey, CACHE_TTL, JSON.stringify
(batchResults[j].embedding));
```

```
      }
    }
  }

  return results as number[][];
}
```

STEP 3: BULLMQ JOB QUEUES

Create file: `src/jobs/queue-config.ts`

```
// src/jobs/queue-config.ts
// BullMQ queue definitions for all background jobs

import { Queue, Worker, QueueEvents } from "bullmq";
import { redis } from "../lib/redis";

const connection = { connection: redis };

// ============================================================
==
// QUEUE DEFINITIONS — One queue per feature area
// ============================================================
==

export const knowledgeQueue = new Queue("knowledge", connecti
on);     // Feature 2: Index pages/tasks/messages/meetings
export const dailyPlanQueue = new Queue("daily-plan", connect
ion);    // Feature 3: Generate daily plans
export const agentQueue = new Queue("agent", connection);
// Feature 4: Execute agent runs
export const riskDetectionQueue = new Queue("risk-detection",
connection); // Feature 5: Scan for project risks
export const meetingQueue = new Queue("meeting", connection);
// Feature 7: Transcribe + summarize meetings
```

```
export const calendarQueue = new Queue("calendar", connectio
n);       // Feature 16: Calendar sync + optimization
export const notificationQueue = new Queue("notification", co
nnection); // Feature 20: Batch + deliver notifications
export const workflowQueue = new Queue("workflow", connectio
n);       // Feature 19: Execute recorded workflows

// ============================================================
==
// DEFAULT JOB OPTIONS — applied to all jobs
// ============================================================
==

export const DEFAULT_JOB_OPTIONS = {
  attempts: 3,                    // retry 3 times on failure
  backoff: {
    type: "exponential" as const,
    delay: 5000,                  // 5s, 10s, 20s
  },
  removeOnComplete: {
    age: 86400,                   // keep completed jobs for
24h
    count: 1000,                  // keep last 1000
  },
  removeOnFail: {
    age: 604800,                  // keep failed jobs for 7 d
ays
  },
};

// ============================================================
==
// RECURRING JOBS (cron schedules)
// ============================================================
==
```

```
export async function setupRecurringJobs() {
  // Feature 2: Full knowledge reindex — every Sunday 2 AM
  await knowledgeQueue.add(
    "full-reindex",
    {},
    {
      repeat: { pattern: "0 2 * * 0" }, // Sunday 2 AM
      ...DEFAULT_JOB_OPTIONS,
    }
  );

  // Feature 5: Risk detection scan — every 6 hours
  await riskDetectionQueue.add(
    "scan-all-projects",
    {},
    {
      repeat: { pattern: "0 */6 * * *" }, // every 6 hours
      ...DEFAULT_JOB_OPTIONS,
    }
  );

  // Feature 16: Calendar sync — every 15 minutes
  await calendarQueue.add(
    "sync-all-calendars",
    {},
    {
      repeat: { pattern: "*/15 * * * *" }, // every 15 min
      ...DEFAULT_JOB_OPTIONS,
    }
  );

  // Feature 20: Process notification batches — 3 times daily
(9am, 1pm, 5pm UTC)
  await notificationQueue.add(
    "batch-digest",
    {},
```

```
    {
      repeat: { pattern: "0 9,13,17 * * *" },
      ...DEFAULT_JOB_OPTIONS,
    }
  );
}
```

STEP 4: SHARED AI UI COMPONENTS

Create file: `src/components/ai/ai-prompt-bar.tsx`

```
// src/components/ai/ai-prompt-bar.tsx
// Universal AI prompt bar — Cmd+J / Ctrl+J from anywhere

"use client";

import { useState, useRef, useEffect, useCallback } from "rea
ct";
import { Sparkles, Send, X, ChevronDown } from "lucide-reac
t";
import { AiStreamingDisplay } from "./ai-streaming-display";
import { AiModelSelector } from "./ai-model-selector";

interface AiPromptBarProps {
  context?: string;          // page content, thread content,
etc.
  contextLabel?: string;     // "This page", "This thread", e
tc.
  placeholder?: string;
  onResult?: (text: string) => void;
  onClose?: () => void;
  feature: string;           // 'writing','search','planner',
etc. — for usage tracking
}
```

```
export function AiPromptBar({
  context,
  contextLabel,
  placeholder = "Ask AI anything...",
  onResult,
  onClose,
  feature,
}: AiPromptBarProps) {
  const [input, setInput] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [streamedText, setStreamedText] = useState("");
  const [isComplete, setIsComplete] = useState(false);
  const [selectedModel, setSelectedModel] = useState<string>
("auto");
  const inputRef = useRef<HTMLTextAreaElement>(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  // Global keyboard shortcut: Cmd+J / Ctrl+J
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if ((e.metaKey || e.ctrlKey) && e.key === "j") {
        e.preventDefault();
        inputRef.current?.focus();
      }
      if (e.key === "Escape" && onClose) {
        onClose();
      }
    };
    document.addEventListener("keydown", handler);
    return () => document.removeEventListener("keydown", hand
ler);
  }, [onClose]);
```

```
  const handleSubmit = useCallback(async () => {
    if (!input.trim() || isLoading) return;
    setIsLoading(true);
    setStreamedText("");
    setIsComplete(false);

    try {
      const response = await fetch("/api/trpc/ai-engine.strea
m", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify({
          prompt: input,
          context: context || undefined,
          model: selectedModel === "auto" ? undefined : selec
tedModel,
          feature,
        }),
      });

      if (!response.body) throw new Error("No response bod
y");

      const reader = response.body.getReader();
      const decoder = new TextDecoder();
      let fullText = "";

      while (true) {
        const { done, value } = await reader.read();
        if (done) break;
        const chunk = decoder.decode(value, { stream: true
});
        fullText += chunk;
        setStreamedText(fullText);
      }
```

```
      setIsComplete(true);
    } catch (error) {
      setStreamedText("Sorry, something went wrong. Please tr
y again.");
      setIsComplete(true);
    } finally {
      setIsLoading(false);
    }
  }, [input, context, selectedModel, feature, isLoading]);

  return (
    <div className="border rounded-lg bg-white shadow-lg p-4
max-w-2xl w-full">
      {/* Context indicator */}
      {contextLabel && (
        <div className="text-xs text-gray-500 mb-2 flex items
-center gap-1">
          <Sparkles className="w-3 h-3" />
          Context: {contextLabel}
        </div>
      )}

      {/* Input area */}
      <div className="flex items-end gap-2">
        <textarea
          ref={inputRef}
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter" && !e.shiftKey) {
              e.preventDefault();
              handleSubmit();
            }
          }}
          placeholder={placeholder}
          className="flex-1 resize-none border rounded-md p-2
```

```
text-sm min-h-[40px] max-h-[120px] focus:outline-none focus:r
ing-2 focus:ring-blue-500"
          rows={1}
        />
        <AiModelSelector value={selectedModel} onChange={setS
electedModel} />
        <button
          onClick={handleSubmit}
          disabled={isLoading || !input.trim()}
          className="p-2 bg-blue-600 text-white rounded-md ho
ver:bg-blue-700 disabled:opacity-50 disabled:cursor-not-allow
ed"
        >
          <Send className="w-4 h-4" />
        </button>
        {onClose && (
          <button onClick={onClose} className="p-2 text-gray-
400 hover:text-gray-600">
            <X className="w-4 h-4" />
          </button>
        )}
      </div>

      {/* Streaming response */}
      {streamedText && (
        <div className="mt-3 border-t pt-3">
          <AiStreamingDisplay text={streamedText} isComplete=
{isComplete} />
          {isComplete && onResult && (
            <div className="mt-2 flex gap-2">
              <button
                onClick={() => onResult(streamedText)}
                className="text-sm px-3 py-1 bg-green-600 tex
t-white rounded hover:bg-green-700"
              >
                ✅ Use this
```

```
              </button>
              <button
                onClick={() => {
                  setStreamedText("");
                  setIsComplete(false);
                }}
                className="text-sm px-3 py-1 bg-gray-200 text
-gray-700 rounded hover:bg-gray-300"
              >
                🔄 Try again
              </button>
              <button
                onClick={() => {
                  setStreamedText("");
                  setIsComplete(false);
                  setInput("");
                }}
                className="text-sm px-3 py-1 text-gray-500 ho
ver:text-gray-700"
              >
                ❌ Discard
              </button>
            </div>
          )}
        </div>
      )}
    </div>
  );
}
```

Create file: `src/components/ai/ai-streaming-display.tsx`

```
// src/components/ai/ai-streaming-display.tsx
// Displays AI-generated text with typing animation

"use client";
```

```
import { useEffect, useRef } from "react";
import { Sparkles, Loader2 } from "lucide-react";

interface AiStreamingDisplayProps {
  text: string;
  isComplete: boolean;
  className?: string;
}

export function AiStreamingDisplay({ text, isComplete, classN
ame = "" }: AiStreamingDisplayProps) {
  const containerRef = useRef<HTMLDivElement>(null);

  // Auto-scroll as text streams in
  useEffect(() => {
    if (containerRef.current) {
      containerRef.current.scrollTop = containerRef.current.s
crollHeight;
    }
  }, [text]);

  return (
    <div className={`relative ${className}`}>
      <div className="flex items-center gap-1 text-xs text-pu
rple-600 mb-1">
        <Sparkles className="w-3 h-3" />
        {isComplete ? "AI Response" : "Generating..."}
        {!isComplete && <Loader2 className="w-3 h-3 animate-s
pin" />}
      </div>
      <div
        ref={containerRef}
        className="text-sm text-gray-800 prose prose-sm max-h
-[300px] overflow-y-auto whitespace-pre-wrap"
      >
```

```
        {text}
        {!isComplete && <span className="animate-pulse">▌</sp
an>}
      </div>
    </div>
  );
}
```

Create file: `src/components/ai/ai-model-selector.tsx`

```
// src/components/ai/ai-model-selector.tsx
// Dropdown for selecting AI model

"use client";

interface AiModelSelectorProps {
  value: string;
  onChange: (model: string) => void;
}

const MODELS = [
  { id: "auto", label: "Auto", description: "Best model for t
he task" },
  { id: "gpt-4o", label: "GPT-4o", description: "Balanced, ve
rsatile" },
  { id: "gpt-4o-mini", label: "GPT-4o Mini", description: "Fa
ster, cheaper" },
  { id: "claude-sonnet", label: "Claude", description: "Stron
g writing" },
  { id: "gemini-pro", label: "Gemini", description: "Data ana
lysis" },
];

export function AiModelSelector({ value, onChange }: AiModelS
electorProps) {
  return (
```

```
    <select
      value={value}
      onChange={(e) => onChange(e.target.value)}
      className="text-xs border rounded px-2 py-1 bg-gray-50
text-gray-600 cursor-pointer hover:bg-gray-100"
    >
      {MODELS.map((m) => (
        <option key={m.id} value={m.id}>
          {m.label}
        </option>
      ))}
    </select>
  );
}
```

Create file: `src/components/ai/ai-diff-view.tsx`

```
// src/components/ai/ai-diff-view.tsx
// Before/After text comparison for AI edits (Feature 1: Writ
ing Assistant)

"use client";

interface AiDiffViewProps {
  original: string;
  modified: string;
  onAccept: () => void;
  onReject: () => void;
}

export function AiDiffView({ original, modified, onAccept, on
Reject }: AiDiffViewProps) {
  return (
    <div className="border rounded-lg overflow-hidden">
      <div className="grid grid-cols-2 divide-x">
        <div className="p-3">
```

```
          <div className="text-xs font-medium text-red-600 mb
-1">Original</div>
          <div className="text-sm text-gray-600 line-through
whitespace-pre-wrap">{original}</div>
        </div>
        <div className="p-3">
          <div className="text-xs font-medium text-green-600
mb-1">AI Suggestion</div>
          <div className="text-sm text-gray-800 bg-green-50 r
ounded p-2 whitespace-pre-wrap">{modified}</div>
        </div>
      </div>
      <div className="flex gap-2 p-2 bg-gray-50 border-t">
        <button
          onClick={onAccept}
          className="text-sm px-3 py-1 bg-green-600 text-whit
e rounded hover:bg-green-700"
        >
          ✅ Accept
        </button>
        <button
          onClick={onReject}
          className="text-sm px-3 py-1 bg-gray-200 text-gray-
700 rounded hover:bg-gray-300"
        >
          ❌ Reject
        </button>
      </div>
    </div>
  );
}
```

Create file: `src/components/ai/ai-suggestion-card.tsx`

```
// src/components/ai/ai-suggestion-card.tsx
// Subtle suggestion card (used in Knowledge Surfacing, Smart
```

```
Notifications, etc.)

"use client";

import { Lightbulb, X } from "lucide-react";

interface AiSuggestionCardProps {
  message: string;
  sources?: { title: string; url: string }[];
  onAccept?: () => void;
  onDismiss?: () => void;
}

export function AiSuggestionCard({ message, sources, onAccep
t, onDismiss }: AiSuggestionCardProps) {
  return (
    <div className="bg-blue-50 border border-blue-200 rounded
-lg p-3 text-sm max-w-md">
      <div className="flex items-start gap-2">
        <Lightbulb className="w-4 h-4 text-blue-600 mt-0.5 fl
ex-shrink-0" />
        <div className="flex-1">
          <p className="text-gray-800">{message}</p>
          {sources && sources.length > 0 && (
            <div className="mt-1 space-y-0.5">
              {sources.map((s, i) => (
                <a key={i} href={s.url} className="text-xs te
xt-blue-600 hover:underline block">
                  {s.title}
                </a>
              ))}
            </div>
          )}
          <div className="mt-2 flex gap-2">
            {onAccept && (
              <button
```

```
                onClick={onAccept}
                className="text-xs px-2 py-0.5 bg-blue-600 te
xt-white rounded hover:bg-blue-700"
              >
                View Source
              </button>
            )}
            {onDismiss && (
              <button
                onClick={onDismiss}
                className="text-xs text-gray-500 hover:text-g
ray-700"
              >
                Dismiss
              </button>
            )}
          </div>
        </div>
        {onDismiss && (
          <button onClick={onDismiss} className="text-gray-40
0 hover:text-gray-600">
            <X className="w-3 h-3" />
          </button>
        )}
      </div>
    </div>
  );
}
```

Create file: `src/components/ai/ai-loading.tsx`

```
// src/components/ai/ai-loading.tsx
// AI processing states

"use client";
```

```
import { Sparkles, Loader2 } from "lucide-react";

interface AiLoadingProps {
  message?: string;
  size?: "sm" | "md" | "lg";
}

export function AiLoading({ message = "AI is thinking...", si
ze = "md" }: AiLoadingProps) {
  const sizeClasses = {
    sm: "text-xs gap-1",
    md: "text-sm gap-2",
    lg: "text-base gap-2",
  };

  return (
    <div className={`flex items-center ${sizeClasses[size]} t
ext-purple-600`}>
      <Sparkles className={size === "sm" ? "w-3 h-3" : "w-4 h
-4"} />
      <span>{message}</span>
      <Loader2 className={`animate-spin ${size === "sm" ? "w-
3 h-3" : "w-4 h-4"}`} />
    </div>
  );
}
```

STEP 5: AI USAGE LOGGING MIDDLEWARE

Create file: `src/server/services/ai-usage-logger.ts`

```
// src/server/services/ai-usage-logger.ts
// Logs all AI usage for analytics. Called after every AI ope
ration.
```

```
import { prisma } from "@/lib/prisma";

interface LogAiUsageParams {
  userId: string;
  workspaceId: string;
  feature: string;
  action: string;
  model: string;
  inputTokens: number;
  outputTokens: number;
  durationMs?: number;
  success?: boolean;
  error?: string;
}

export async function logAiUsage(params: LogAiUsageParams): P
romise<void> {
  try {
    await prisma.aiUsageLog.create({
      data: {
        userId: params.userId,
        workspaceId: params.workspaceId,
        feature: params.feature,
        action: params.action,
        model: params.model,
        inputTokens: params.inputTokens,
        outputTokens: params.outputTokens,
        durationMs: params.durationMs,
        success: params.success ?? true,
        error: params.error,
      },
    });
  } catch (err) {
    // Never throw from logging — don't break the feature bec
ause logging failed
    console.error("Failed to log AI usage:", err);
```

```
  }
}
```

STEP 6: KNOWLEDGE INDEXING PIPELINE (Feature 2 — Backend Only)

Create file: `src/server/services/knowledge-indexer.ts`

```
// src/server/services/knowledge-indexer.ts
// Indexes pages, tasks, messages, and meetings for AI search
(Feature 2)

import { prisma } from "@/lib/prisma";
import { getEmbedding, getEmbeddingBatch } from "@/lib/ai/emb
eddings";

// ============================================================
==
// INDEX A SINGLE PAGE
// ============================================================
==

export async function indexPage(pageId: string, workspaceId:
string): Promise<void> {
  const page = await prisma.page.findUnique({
    where: { id: pageId },
    include: { blocks: true }, // adjust to your actual page/
block schema
  });
  if (!page) return;

  // Extract plain text from all blocks
  const content = extractTextFromBlocks(page.blocks);
  if (!content.trim()) return;
```

```
  // Generate embedding
  const embedding = await getEmbedding(content.slice(0, 800
0)); // limit to ~8k chars for embedding

  // Chunk long content for better search precision
  const chunks = content.length > 2000 ? chunkText(content, 5
00) : null;

  // Upsert into knowledge index
  await prisma.$executeRaw`
    INSERT INTO "KnowledgeIndex" (
      id, "workspaceId", "sourceType", "sourceId", title, con
tent,
      "contentChunks", embedding, metadata, "accessLevel",
      "lastIndexedAt", "isStale", "createdAt", "updatedAt"
    ) VALUES (
      gen_random_uuid(), ${workspaceId}, 'page', ${pageId},
      ${page.title || "Untitled"}, ${content},
      ${chunks ? JSON.stringify(chunks) : null}::jsonb,
      ${embedding}::vector,
      ${JSON.stringify({
        authorId: page.createdBy,
        createdDate: page.createdAt,
        lastModifiedDate: page.updatedAt,
      })}::jsonb,
      'workspace',
      NOW(), false, NOW(), NOW()
    )
    ON CONFLICT ("sourceId") DO UPDATE SET
      title = EXCLUDED.title,
      content = EXCLUDED.content,
      "contentChunks" = EXCLUDED."contentChunks",
      embedding = EXCLUDED.embedding,
      metadata = EXCLUDED.metadata,
      "lastIndexedAt" = NOW(),
      "isStale" = false,
```

```
      "updatedAt" = NOW()
  `;
}

// ============================================================
==
// INDEX MESSAGES IN BATCH (every 5 minutes)
// ============================================================
==

export async function indexMessageBatch(
  messages: { id: string; content: string; channelId: string;
authorId: string; authorName: string; createdAt: Date }[],
  workspaceId: string
): Promise<void> {
  if (messages.length === 0) return;

  const texts = messages.map((m) => m.content);
  const embeddings = await getEmbeddingBatch(texts);

  for (let i = 0; i < messages.length; i++) {
    const msg = messages[i];
    await prisma.$executeRaw`
      INSERT INTO "KnowledgeIndex" (
        id, "workspaceId", "sourceType", "sourceId", "parentS
ourceId",
        title, content, embedding, metadata, "accessLevel",
        "lastIndexedAt", "isStale", "createdAt", "updatedAt"
      ) VALUES (
        gen_random_uuid(), ${workspaceId}, 'message', ${msg.i
d}, ${msg.channelId},
        ${msg.content.slice(0, 100)}, ${msg.content},
        ${embeddings[i]}::vector,
        ${JSON.stringify({
          authorId: msg.authorId,
          authorName: msg.authorName,
```

```
          channelId: msg.channelId,
          createdDate: msg.createdAt,
        })}::jsonb,
        'channel_members',
        NOW(), false, NOW(), NOW()
      )
      ON CONFLICT ("sourceId") DO NOTHING
    `;
  }
}

// ============================================================
==
// INDEX A TASK
// ============================================================
==

export async function indexTask(taskId: string, workspaceId:
string): Promise<void> {
  const task = await prisma.task.findUnique({ where: { id: ta
skId } });
  if (!task) return;

  const content = `${task.title}\n${task.description || ""}`;
  const embedding = await getEmbedding(content);

  await prisma.$executeRaw`
    INSERT INTO "KnowledgeIndex" (
      id, "workspaceId", "sourceType", "sourceId",
      title, content, embedding, metadata, "accessLevel",
      "lastIndexedAt", "isStale", "createdAt", "updatedAt"
    ) VALUES (
      gen_random_uuid(), ${workspaceId}, 'task', ${taskId},
      ${task.title}, ${content},
      ${embedding}::vector,
      ${JSON.stringify({
```

```
        authorId: task.createdBy,
        createdDate: task.createdAt,
        lastModifiedDate: task.updatedAt,
      })}::jsonb,
      'workspace',
      NOW(), false, NOW(), NOW()
    )
    ON CONFLICT ("sourceId") DO UPDATE SET
      title = EXCLUDED.title,
      content = EXCLUDED.content,
      embedding = EXCLUDED.embedding,
      metadata = EXCLUDED.metadata,
      "lastIndexedAt" = NOW(),
      "isStale" = false,
      "updatedAt" = NOW()
  `;
}

// ============================================================
==
// HELPER FUNCTIONS
// ============================================================
==

function extractTextFromBlocks(blocks: any[]): string {
  // Adapt this to your actual block structure
  return blocks
    .map((block) => {
      if (typeof block.content === "string") return block.con
tent;
      if (block.content?.text) return block.content.text;
      if (block.type === "heading") return block.content?.tex
t || "";
      if (block.type === "paragraph") return block.content?.t
ext || "";
      if (block.type === "list") {
```

```
        return (block.content?.items || []).map((item: any) =
> item.text || "").join("\n");
      }
      return "";
    })
    .filter(Boolean)
    .join("\n");
}

function chunkText(text: string, chunkSize: number): { chunkI
ndex: number; text: string }[] {
  const chunks: { chunkIndex: number; text: string }[] = [];
  const sentences = text.split(/(?<=[.!?])\s+/);
  let current = "";
  let index = 0;

  for (const sentence of sentences) {
    if (current.length + sentence.length > chunkSize && curre
nt.length > 0) {
      chunks.push({ chunkIndex: index++, text: current.trim()
});
      current = "";
    }
    current += sentence + " ";
  }
  if (current.trim()) {
    chunks.push({ chunkIndex: index, text: current.trim() });
  }

  return chunks;
}
```

SPRINT 1 VERIFICATION CHECKLIST

After building everything above, verify:

```
□ npx prisma migrate dev --name add_all_ai_features → runs wi
thout errors
□ npx prisma generate → creates client successfully
□ Import aiService and call generateText("Hello") → returns r
esponse
□ Import aiService and call generateEmbedding("test") → retur
ns 1536-dimension vector
□ BullMQ queues initialize without errors (check Redis connec
tion)
□ All shared components render (create a test page that impor
ts each one)
□ TypeScript compilation: npx tsc --noEmit → zero errors
□ Redis connection works: import redis, call redis.ping() →
"PONG"
```

Once all 8 checks pass, proceed to Sprint 2.